



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

❖ Hashing

⇒ **Definition :-** Hashing is a technique used to store and retrieve data efficiently by converting a key into an array index using a hash function. It provides average $O(1)$ time complexity for searching, insertion, and deletion.

✓ Need for Hashing

- Fast searching of records
- Fast insertion and deletion
- Avoids linear searching
- Efficient for large data sets

➤ Basic Terminology

1. Key

A value used to identify a record.

Example: Student ID = 105

2. Hash Function

A mathematical function that converts a key into an index.

Formula:

$$h(k) = k \bmod m$$

where:

- **k** = key
- **m** = size of hash table

Example:

- Table size = 10
- Key = 57

$$h(57) = 57 \bmod 10 = 7$$

Store the record at **index 7**.

3. Hash Value

The index produced by the hash function.

Example:

- Key = 57
- Hash Value = 7



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

4. Hash Table

An array in which records are stored according to the hash value.

Example:

Index	Data
0	
1	
2	22
3	
4	34
5	
6	
7	57
8	18
9	

➤ Working of Hashing

Given keys:

22, 57, 34, 18

Hash Function:

$$h(k)=k \bmod 10$$

Key	Calculation	Index
22	$22 \bmod 10 = 2$	2
57	$57 \bmod 10 = 7$	7
34	$34 \bmod 10 = 4$	4
18	$18 \bmod 10 = 8$	8

Searching key 57:

- Compute hash value
- Go directly to index 7
- Record found immediately

❖ Characteristics of a Good Hash Function

A good hash function should:

- Be simple to compute
- Produce unique hash values as much as possible
- Distribute keys uniformly
- Minimize collisions
- Execute quickly



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

➤ Advantages of Hashing

- Very fast searching
- Fast insertion
- Fast deletion
- Efficient memory access
- Suitable for large databases

➤ Disadvantages of Hashing

- Collision may occur
- Performance decreases if many collisions occur
- Choosing a good hash function is difficult
- Extra techniques are needed to resolve collisions

➤ Applications of Hashing

- Database indexing
- Dictionaries
- Compiler symbol tables
- Password verification
- Caching
- File systems
- Routing tables
- Blockchain
- Student and employee record management

✓ Symbol Table

⇒ **Definition:-** A Symbol Table is an abstract data structure that stores key–value pairs, where each key is unique and is associated with a value. It supports efficient insertion, searching, updating, and deletion of records.

Example

Roll No. (Key)	Student Name (Value)
101	Amit
102	Neha
103	Riya
104	Raj

Searching **Key = 103** returns **Riya**.



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

Operations on Symbol Table

1. Insert

Adds a new key-value pair.

Example:

Insert(101, "Amit")

2. Search

Finds the value associated with a key.

Example:

Search(103)

Output:

Riya

3. Update

Changes the value associated with a key.

Example:

Update(103, "Priya")

4. Delete

Removes a key-value pair.

Example:

Delete(102)

Symbol Table Example

Before update:

Key	Value
101	Amit
102	Neha
103	Riya

After:

Update(103, "Priya")

Key	Value
101	Amit
102	Neha
103	Priya



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

Implementation of Symbol Table

A symbol table can be implemented using:

- Array
- Linked List
- Binary Search Tree (BST)
- AVL Tree
- Red-Black Tree
- Hash Table (Most efficient for average-case operations)

Advantages of Symbol Table

- Fast searching
- Easy insertion and deletion
- Efficient key-value storage
- Dynamic updates
- Widely used in programming languages and databases

Applications of Symbol Table

- Compiler (stores variable names and attributes)
- Database Management System (DBMS)
- Dictionary
- Phone directory
- Library Management System
- Student Management System
- Employee Record System

Difference Between Hashing and Symbol Table

Hashing	Symbol Table
Technique used to map keys to indices	Abstract data structure that stores key-value pairs
Uses a hash function	Can be implemented using hashing, BST, arrays, etc.
Focuses on fast storage and retrieval	Focuses on managing key-value mappings
Used to implement hash tables	Can be implemented using a hash table

Exam Key Points

- **Hashing** maps a key to an array index using a **hash function**.
- A **hash table** stores data at the computed index.
- A **good hash function** distributes keys uniformly and minimizes collisions.
- **Symbol Table** stores **unique key-value pairs**.
- Main operations: **Insert, Search, Update, Delete**.



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

❖ Hash Function Using Division Method (Data Structures)

⇒ **Definition :-** The Division Method is the simplest and most commonly used hashing technique. In this method, the key is divided by the size of the hash table, and the remainder is used as the hash value (index).

Formula

$$h(k) = k \text{ mod } m$$

where:

- **$h(k)$** = Hash function
- **k** = Key
- **m** = Size of the hash table
- **mod** = Modulus (remainder) operator

Algorithm

1. Read the key **k** .
2. Determine the hash table size **m** .
3. Calculate the hash value using:

$$h(k) = k \text{ mod } m$$

4. Store the record at the computed index.

Example 1

Hash Table Size (**m**) = 10

Keys: 25, 37, 42, 58, 91

Hash Function:

$$h(k) = k \text{ mod } 10$$

Key (k)	Calculation	Hash Value (Index)
25	$25 \text{ mod } 10 = 5$	5
37	$37 \text{ mod } 10 = 7$	7
42	$42 \text{ mod } 10 = 2$	2
58	$58 \text{ mod } 10 = 8$	8
91	$91 \text{ mod } 10 = 1$	1



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

Hash Table

Index	Data
0	
1	91
2	42
3	
4	
5	25
6	
7	37
8	58
9	

Example 2 (Collision)

Hash Table Size = 10

Keys: 25, 35, 45

Hash Function:

$$h(k) = k \bmod 10$$

Key	Calculation	Index
25	$25 \bmod 10 = 5$	5
35	$35 \bmod 10 = 5$	5
45	$45 \bmod 10 = 5$	5

All three keys map to **Index 5**, causing a **collision**.

Choosing a Good Table Size

For better distribution of keys:

- Prefer a **prime number** as the table size.
- Avoid powers of 2 (when possible).
- A prime table size reduces collisions.

Example: Use 11, 13, 17, 19 instead of 10, 20, 50.

• Advantages of Division Method

- ✓ Very simple to implement.
- ✓ Fast computation.
- ✓ Requires only one modulus operation.
- ✓ Suitable for small and medium-sized hash tables.



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

- ✓ Provides good performance when a prime table size is used.

• Disadvantages of Division Method

- ✓ May produce many collisions if the table size is chosen poorly.
- ✓ Performance depends on the choice of m .
- ✓ Not suitable when keys follow a specific pattern.

• Applications

- ✓ Hash Tables
- ✓ Symbol Tables
- ✓ Database Indexing
- ✓ Dictionary Implementation
- ✓ Compiler Design
- ✓ Caching Systems

❖ Collision Resolution Technique – Chaining (Separate Chaining)

⇒ **Definition :-** Chaining (also called **Separate Chaining** or **Open Hashing**) is a collision resolution technique in which each location (index) of the hash table stores a **linked list** of all keys that hash to the same index.

When multiple keys map to the same index, they are inserted into the linked list associated with that index instead of replacing the existing key.

• Why Chaining is Needed?

When two or more keys produce the same hash value, a **collision** occurs.

Instead of searching for another empty location, **chaining stores all colliding keys in a linked list at the same index.**

• Working of Chaining

1. Compute the hash value using the hash function.
2. If the index is empty, insert the key.
3. If the index already contains a key, create (or use) a linked list at that index.
4. Insert the new key into the linked list.



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

Example :

Keys : 25, 35, 15, 42, 52, 62

Hash Function : $h(k) = k \bmod N$

$N = 10$

$h(25) = 25 \bmod 10 = 5$

$h(35) = 35 \bmod 10 = 5$

$h(15) = 15 \bmod 10 = 5$

$h(42) = 42 \bmod 10 = 2$

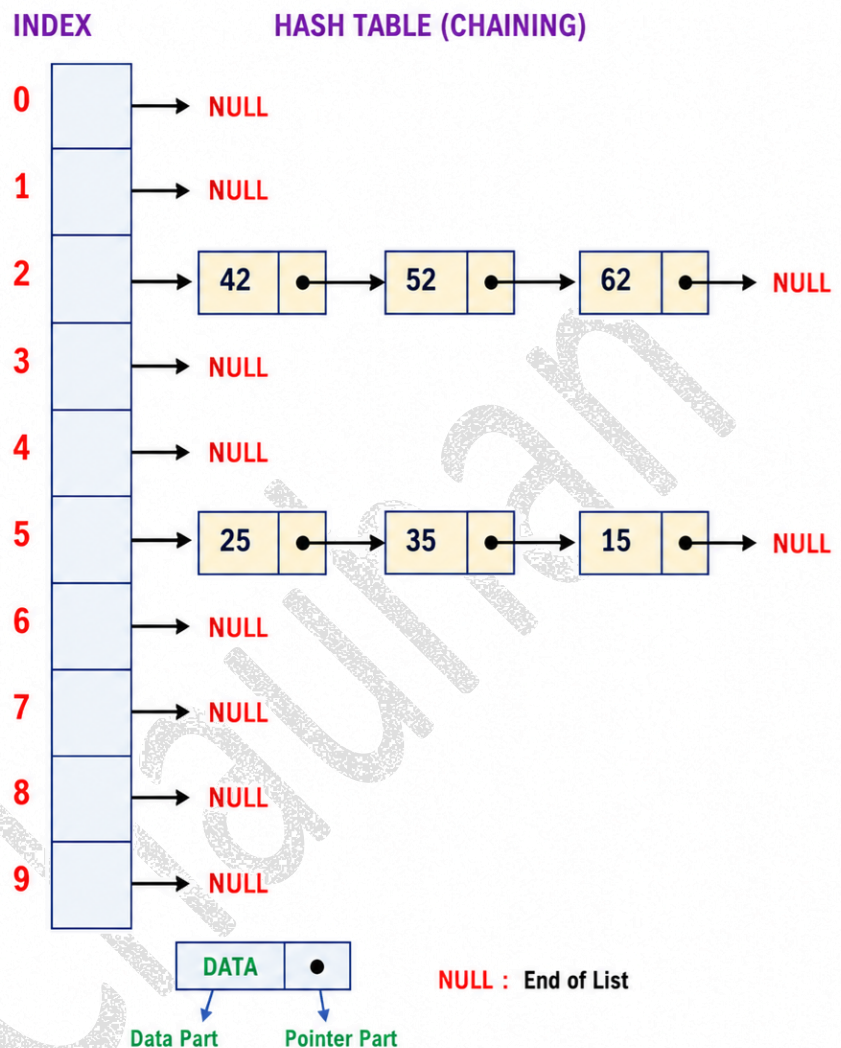
$h(52) = 52 \bmod 10 = 2$

$h(62) = 62 \bmod 10 = 2$

Result :

Keys 25, 35, 15 → Index 5

Keys 42, 52, 62 → Index 2



• Advantages of Chaining

- ✓ Simple and easy to implement.
- ✓ Easy insertion and deletion.
- ✓ Hash table never becomes completely full.
- ✓ Handles many collisions efficiently.
- ✓ Performance degrades gradually as the number of collisions increases.

• Disadvantages of Chaining

- ✓ Requires extra memory for linked lists.
- ✓ Searching becomes slower if the linked list becomes long.
- ✓ Poor cache performance because linked-list nodes are not stored contiguously.
- ✓ Dynamic memory allocation is required.



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

- **Applications**

- ✓ Symbol Tables
- ✓ Dictionary Implementation
- ✓ Database Indexing
- ✓ Compiler Design
- ✓ Caching Systems
- ✓ File Organization

- ❖ **Collision Resolution Technique – Linear Probing**

⇒ **Definition :-** **Linear Probing** is an **Open Addressing** collision resolution technique in which, if a collision occurs, the next available location in the hash table is searched **sequentially** until an empty slot is found. It checks positions one by one in a linear manner.

Probe Function

If a collision occurs,

Formula: $h(k, i) = [h'(k) + i] \bmod n$

- $h'(k)$ = Any Hash Function
- i = Number of probes (0,1,2,...)
- n = Size of hash table
- $h'(k) = k \bmod n$

Algorithm

1. Compute the hash value.
2. If the location is empty, insert the key.
3. If occupied, move to the next location.
4. Continue until an empty location is found.

- **Advantages**

- ✓ Easy to implement.
- ✓ No extra memory is required.
- ✓ All elements are stored inside the hash table.
- ✓ Good performance when the load factor is low.



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

• Disadvantages

- ✓ Primary Clustering occurs (long consecutive occupied locations).
- ✓ Performance decreases as the table becomes full.
- ✓ Deletion is more difficult than in chaining.
- ✓ Requires empty slots to insert new elements.

Example : 176, 75, 37, 56, 79, 154, 10, 99

Hash Function : $h(k) = k \bmod N$ (Linear Probing)

$N = 10$

1) **Key = 176**

$$h(176, 0) = (176 \bmod 10 + 0) \bmod 10 \\ = (6 + 0) \bmod 10 = 6$$

2) **Key = 75**

$$h(75, 0) = (75 \bmod 10 + 0) \bmod 10 \\ = (5 + 0) \bmod 10 = 5$$

3) **Key = 37**

$$h(37, 0) = (37 \bmod 10 + 0) \bmod 10 \\ = (7 + 0) \bmod 10 = 7$$

4) **Key = 56**

$$h(56, 0) = (56 \bmod 10 + 0) \bmod 10 = 6 \rightarrow \text{Occupied} \\ h(56, 1) = (56 \bmod 10 + 1) \bmod 10 = 7 \rightarrow \text{Occupied} \\ h(56, 2) = (56 \bmod 10 + 2) \bmod 10 = 8 \rightarrow \text{Empty} \\ \text{Insert 56 at index 8.}$$

5) **Key = 79**

$$h(79, 0) = (79 \bmod 10 + 0) \bmod 10 = 9 \rightarrow \text{Empty} \\ \text{Insert 79 at index 9.}$$

6) **Key = 154**

$$h(154, 0) = (154 \bmod 10 + 0) \bmod 10 = 4 \rightarrow \text{Empty} \\ \text{Insert 154 at index 4.}$$

7) **Key = 10**

$$h(10, 0) = (10 \bmod 10 + 0) \bmod 10 = 0 \rightarrow \text{Empty} \\ \text{Insert 10 at index 0.}$$

8) **Key = 99**

$$h(99, 0) = (99 \bmod 10 + 0) \bmod 10 = 9 \rightarrow \text{Occupied} \\ h(99, 1) = (99 \bmod 10 + 1) \bmod 10 = 0 \rightarrow \text{Occupied} \\ h(99, 2) = (99 \bmod 10 + 2) \bmod 10 = 1 \rightarrow \text{Empty} \\ \text{Insert 99 at index 1.}$$

1)

Index	0	1	2	3	4	5	6	7	8	9
							176			

2)

Index	0	1	2	3	4	5	6	7	8	9
						75	176			

3)

Index	0	1	2	3	4	5	6	7	8	9
						75	176	37		

4)

Index	0	1	2	3	4	5	6	7	8	9
						75	176	37	56	

5)

Index	0	1	2	3	4	5	6	7	8	9
					75	176	37	56	79	

6)

Index	0	1	2	3	4	5	7	8	8	9
				154	75	176	37	56	79	

7)

Index	0	1	2	?	4	5	6	7	8	9
	10				154	75	176	37	56	79

8)

Index	0	1	2	3	4	5	6	7	8	9
	10	99			154	75	176	37	56	79

❖ Quadratic Probing

⇒ **Definition :-** Quadratic Probing is an **open addressing** collision resolution technique.

When collision occurs, it does not check the next location one by one.

Instead, it checks locations by adding square values:

Formula: $h(k, i) = [h'(k) + i^2] \bmod n$

- $h'(k)$ = Any Hash Function
- k = key



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

- i = probe number
- N = hash table size
- $h'(k) = k \bmod n$

Quadratic Probing – Formula

Formula : $h(k, i) = (k \bmod N + i^2) \bmod N$

For collision:

i	0	1	2	3
i^2	$0^2 = 0$	$1^2 = 1$	$2^2 = 4$	$3^2 = 9$

So checking positions are:

$h(k)$

,

$h(k) + 1^2$

,

$h(k) + 2^2$

,

$h(k) + 3^2$

Advantages

- ✓ Reduces primary clustering.
- ✓ Better than linear probing.
- ✓ Does not require linked list.
- ✓ All data is stored inside the hash table.

Disadvantages

- ✓ Secondary clustering may occur.
- ✓ Empty position may not always be found.
- ✓ Deletion is difficult.
- ✓ Works better when table size is prime.



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

Quadratic Probing - Detailed Example

Formula: $h(k,i) = (k \bmod 10 + i^2) \bmod 10$

Keys: 176, 75, 37, 56, 79, 154, 10, 99

Step 1: Insert 176

Hash Calculation: $176 \bmod 10 = 6$

Quadratic Probing:
No collision.

Result: Insert at Index 6

Index	0	1	2	3	4	5	6	7	8	9
Value							176			

Step 2: Insert 75

Hash Calculation: $75 \bmod 10 = 5$

Quadratic Probing:
No collision.

Result: Insert at Index 5

Index	0	1	2	3	4	5	6	7	8	9
Value						75	176			

Step 3: Insert 37

Hash Calculation: $37 \bmod 10 = 7$

Quadratic Probing:
No collision.

Result: Insert at Index 7

Index	0	1	2	3	4	5	6	7	8	9
Value						75	176	37		



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

Step 4: Insert 56

Hash Calculation: $56 \bmod 10 = 6$

Quadratic Probing:

$i = 0 \rightarrow (56 \bmod 10 + 0^2) \bmod 10 = (6 + 0) \bmod 10 = 6$ (Occupied)

$i = 1 \rightarrow (56 \bmod 10 + 1^2) \bmod 10 = (6 + 1) \bmod 10 = 7$ (Occupied)

$i = 2 \rightarrow (56 \bmod 10 + 2^2) \bmod 10 = (6 + 4) \bmod 10 = 0$ (Empty)

Result: Insert at Index 0

Index	0	1	2	3	4	5	6	7	8	9
Value	56					75	176	37		

Step 5: Insert 79

Hash Calculation: $79 \bmod 10 = 9$

Quadratic Probing:

No collision.

Result: Insert at Index 9

Index	0	1	2	3	4	5	6	7	8	9
Value	56					75	176	37		79

Step 6: Insert 154

Hash Calculation: $154 \bmod 10 = 4$

Quadratic Probing:

No collision.

Result: Insert at Index 4

Index	0	1	2	3	4	5	6	7	8	9
Value	56				154	75	176	37		79



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

Step 7: Insert 10

Hash Calculation: $10 \bmod 10 = 0$

Quadratic Probing:

$i = 0 \rightarrow (10 \bmod 10 + 0^2) \bmod 10 = (0 + 0) \bmod 10 = 0$ (Occupied)

$i = 1 \rightarrow (10 \bmod 10 + 1^2) \bmod 10 = (0 + 1) \bmod 10 = 1$ (Empty)

Result: Insert at Index 1

Index	0	1	2	3	4	5	6	7	8	9
Value	56	10			154	75	176	37		79

Step 8: Insert 99

Hash Calculation: $99 \bmod 10 = 9$

Quadratic Probing:

$i = 0 \rightarrow (99 \bmod 10 + 0^2) \bmod 10 = (9 + 0) \bmod 10 = 9$ (Occupied)

$i = 1 \rightarrow (99 \bmod 10 + 1^2) \bmod 10 = (9 + 1) \bmod 10 = 0$ (Occupied)

$i = 2 \rightarrow (99 \bmod 10 + 2^2) \bmod 10 = (9 + 4) \bmod 10 = 3$ (Empty)

Result: Insert at Index 3

Index	0	1	2	3	4	5	6	7	8	9
Value	56	10		99	154	75	176	37		79

❖ Double Hashing

⇒ **Definition:-** Double Hashing is an **open addressing** collision resolution technique that uses **two hash functions**.

- The **first hash function** determines the initial index.
- The **second hash function** determines the step size (jump distance) when a collision occurs.

Since the step size varies for different keys, Double Hashing greatly **reduces clustering** compared to Linear and Quadratic Probing.

• Advantages

- ✓ Reduces primary clustering.
- ✓ Reduces secondary clustering.
- ✓ Better distribution than Linear and Quadratic Probing.
- ✓ Efficient searching and insertion.



Semester: 2

Subject : Data Structures using Java

Chapter : Hashing

Disadvantages

- ✓ Requires two hash functions.
- ✓ Slightly more complex implementation.
- ✓ Choosing a good secondary hash function is important.

DOUBLE HASHING

Double hashing is an open addressing technique in which two hash functions are used. The second hash function decides the step size (jump).

FORMULA

$$h(k, i) = [h'(k) + i \times h''(k)] \bmod n$$

where, $h'(k)$: Primary hash function
 $h''(k)$: Secondary hash function
 i : Probe number (0, 1, 2, ...)
 n : Size of hash table

EXAMPLE

Consider a hash table of size $n = 10$.

Use the following hash functions:

Primary hash function : $h'(k) = k \bmod 10$

Secondary hash function : $h''(k) = 7 - (k \bmod 7)$

Insert the following keys : 25, 35, 15, 42, 52, 62

Step	Key	Calculations	Hash Table (n = 10)																						
Step 1 Insert 25		$h'(25) = 25 \bmod 10 = 5 \rightarrow$ Index 5 is empty. Insert 25 at index 5.	<table><tr><td>Index</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td></tr><tr><td>Value</td><td>-</td><td>-</td><td>-</td><td>-</td><td>-</td><td>25</td><td>-</td><td>-</td><td>-</td><td>-</td></tr></table>	Index	0	1	2	3	4	5	6	7	8	9	Value	-	-	-	-	-	25	-	-	-	-
Index	0	1	2	3	4	5	6	7	8	9															
Value	-	-	-	-	-	25	-	-	-	-															
Step 2 Insert 35		$h'(25) = 35 \bmod 10 = 5 \rightarrow$ Collision at index 5. $h''(35) = 7 - (35 \bmod 7) = 7 - 0 = 7$ $i = 0 \rightarrow h(35, 0) = (35 \bmod 10 + 0 \times 7) \bmod 10 = 5$ (Occupied) $i = 1 \rightarrow h(35, 1) = (35 \bmod 10 + 1 \times 7) \bmod 10 = 2$ (Empty) Insert 35 at index 2.	<table><tr><td>Index</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td></tr><tr><td>Value</td><td>-</td><td>-</td><td>35</td><td>-</td><td>-</td><td>25</td><td>-</td><td>-</td><td>-</td><td>-</td></tr></table>	Index	0	1	2	3	4	5	6	7	8	9	Value	-	-	35	-	-	25	-	-	-	-
Index	0	1	2	3	4	5	6	7	8	9															
Value	-	-	35	-	-	25	-	-	-	-															
Step 3 Insert 15		$h'(15) = 15 \bmod 10 = 5 \rightarrow$ Collision. $h''(15) = 7 - (15 \bmod 7) = 7 - 1 = 6$ $i = 0 \rightarrow h(15, 0) = (15 \bmod 10 + 0 \times 6) \bmod 10 = 5$ (Occupied) $i = 1 \rightarrow h(15, 1) = (15 \bmod 10 + 1 \times 6) \bmod 10 = 1$ (Empty) Insert 15 at index 1.	<table><tr><td>Index</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td></tr><tr><td>Value</td><td>-</td><td>15</td><td>35</td><td>-</td><td>-</td><td>25</td><td>-</td><td>-</td><td>-</td><td>-</td></tr></table>	Index	0	1	2	3	4	5	6	7	8	9	Value	-	15	35	-	-	25	-	-	-	-
Index	0	1	2	3	4	5	6	7	8	9															
Value	-	15	35	-	-	25	-	-	-	-															
Step 4 Insert 42		$h'(42) = 42 \bmod 10 = 2 \rightarrow$ Collision. $h''(42) = 7 - (42 \bmod 7) = 7 - 0 = 7$ $i = 0 \rightarrow h(42, 0) = (42 \bmod 10 + 0 \times 7) \bmod 10 = 2$ (Occupied) $i = 1 \rightarrow h(42, 1) = (42 \bmod 10 + 1 \times 7) \bmod 10 = 9$ (Empty) Insert 42 at index 9.	<table><tr><td>Index</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td></tr><tr><td>Value</td><td>-</td><td>15</td><td>35</td><td>-</td><td>-</td><td>25</td><td>-</td><td>-</td><td>-</td><td>42</td></tr></table>	Index	0	1	2	3	4	5	6	7	8	9	Value	-	15	35	-	-	25	-	-	-	42
Index	0	1	2	3	4	5	6	7	8	9															
Value	-	15	35	-	-	25	-	-	-	42															
Step 5 Insert 52		$h'(52) = 52 \bmod 10 = 2 \rightarrow$ Collision. $h''(52) = 7 - (52 \bmod 7) = 7 - 3 = 4$ $i = 0 \rightarrow h(52, 0) = (52 \bmod 10 + 0 \times 4) \bmod 10 = 2$ (Occupied) $i = 1 \rightarrow h(52, 1) = (52 \bmod 10 + 1 \times 4) \bmod 10 = 6$ (Empty) Insert 52 at index 6.	<table><tr><td>Index</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td></tr><tr><td>Value</td><td>-</td><td>15</td><td>35</td><td>-</td><td>-</td><td>25</td><td>52</td><td>-</td><td>-</td><td>42</td></tr></table>	Index	0	1	2	3	4	5	6	7	8	9	Value	-	15	35	-	-	25	52	-	-	42
Index	0	1	2	3	4	5	6	7	8	9															
Value	-	15	35	-	-	25	52	-	-	42															
Step 6 Insert 62		$h'(62) = 62 \bmod 10 = 2 \rightarrow$ Collision. $h''(62) = 7 - (62 \bmod 7) = 7 - 6 = 1$ $i = 0 \rightarrow h(62, 0) = (62 \bmod 10 + 0 \times 1) \bmod 10 = 2$ (Occupied) $i = 1 \rightarrow h(62, 1) = (62 \bmod 10 + 1 \times 1) \bmod 10 = 3$ (Empty) Insert 62 at index 3.	<table><tr><td>Index</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td></tr><tr><td>Value</td><td>-</td><td>15</td><td>35</td><td>62</td><td>-</td><td>25</td><td>52</td><td>-</td><td>-</td><td>42</td></tr></table>	Index	0	1	2	3	4	5	6	7	8	9	Value	-	15	35	62	-	25	52	-	-	42
Index	0	1	2	3	4	5	6	7	8	9															
Value	-	15	35	62	-	25	52	-	-	42															

FINAL HASH TABLE

Index	0	1	2	3	4	5	6	7	8	9
Value	-	15	35	62	-	25	52	-	-	42

Notes:

- Two hash functions are used.
- The step size is decided by the secondary hash function.
- It reduces both primary and secondary clustering.